

Claire AI Subsystem Dependency Graph

****CLAIRE AI -- SUBSYSTEM DEPENDENCY GRAPH (Full System Architecture Map Medium-Wide Layout Unicode Symbols Only Export-Ready)****

1. Overview

This document defines the complete subsystem dependency architecture for Claire AI, including:

- Engines
- Layers
- Pipelines
- Safety systems
- Mode-switching logic
- Data flow boundaries
- Isolation rules
- Fusion layers

This is the artifact that demonstrates Claire is a real, implementable, multi-engine system with clean separation of concerns and governed interactions.

2. High-Level Subsystem Map (Unicode Diagram)

```
CLAIRE AI -- SYSTEM Deterministic Connected Engine Intelligence Engine
Semantic Layer Ingestion Layer Concept Graph Signal Processing Engine
Engine Breakthrough Live Scoring Engine Engine v v Deterministic Live
Semantic Semantic Maps Overlays v v Hybrid Fusion Intelligence Layer v
Breakthrough Detection Engine v Scoring & Validation Layer v Portfolio
Candidate Object (PCO) v Portfolio Creator Engine v Portfolio Engine
(Clusters) v Portfolio Dashboard Safety & Governance Layer Safety Matrix
Mode-Switching Controller Redline Detection Mode State Manager Compliance
Engine Isolation Enforcement Engine Activation Logic
```

3. Subsystem Dependency Breakdown

Below is the full dependency hierarchy, showing which subsystems rely on which others.

3.1 Deterministic Engine Dependencies

The Deterministic Engine depends on:

Semantic Extraction Layer

Concept Graph Engine

Deterministic Embeddings

Internal Logic Models

Patent-Safe Knowledge Base

It has no external dependencies.

3.2 Connected Intelligence Engine Dependencies

The Connected Engine depends on:

Live Ingestion Layer

Signal Processing Engine

External Data Feeds (governed)

Live Scoring Models

Safety Matrix (for ingestion gating)

It cannot run without the Safety Layer.

3.3 Hybrid Fusion Layer Dependencies

The Hybrid Layer depends on:

Deterministic Semantic Maps

Live Semantic Overlays

Fusion Logic

Mode-Switching Controller

Safety Layer (for dual-pipeline governance)

This is the most complex dependency node.

3.4 Breakthrough Detection Engine Dependencies

Breakthrough Detection depends on:

Deterministic Engine (always)

Connected Engine (if active)

Hybrid Fusion Layer (if in hybrid mode)

Scoring Layer

Safety Layer (for redline checks)

This engine is the central convergence point.

3.5 Scoring & Validation Layer Dependencies

Depends on:

Breakthrough Engine

Market Engine

Competitive Engine

Timing Engine

Acquirer Fit Engine

Safety Layer (for compliance scoring)

This layer determines whether a breakthrough becomes a PCO.

3.6 Portfolio Candidate Object (PCO) Dependencies

Depends on:

Breakthrough Object

Scoring Layer

Safety Layer (for governance flags)

PCO creation is blocked if safety flags are triggered.

3.7 Portfolio Creator Engine Dependencies

Depends on:

PCO

Technical Engine

Market Engine

Strategic Engine

Risk Engine

Clustering Engine

This engine produces the Portfolio Item (PI).

3.8 Portfolio Engine Dependencies

Depends on:

Portfolio Items

Clustering Engine

Theme Engine

Adjacency Mapping Engine

This engine powers the Portfolio Dashboard.

3.9 Safety & Governance Layer Dependencies

Depends on:

Safety Matrix

Redline Detection Engine

Compliance Engine

Audit Logging System

Mode-Switching Controller

This layer governs all other layers.

3.10 Mode-Switching Controller Dependencies

Depends on:

Mode State Object

Safety Layer

Engine Activation Logic

Isolation Enforcement Logic

This controller determines which engines are active.

4. Dependency Rules (Critical for Acquisition)

****Rule 1 -- Deterministic Engine must run in isolation****

No external dependencies allowed.

****Rule 2 -- Connected Engine requires Safety Layer approval****

No ingestion without safety gating.

****Rule 3 -- Hybrid Mode requires dual-pipeline activation****

Both engines must be active and governed.

****Rule 4 -- Breakthrough Engine is mode-agnostic but dependency-aware****

It adapts based on active engines.

****Rule 5 -- Portfolio Creator cannot run without validated PCO****

Safety and scoring must approve.

****Rule 6 -- Safety Layer overrides all other subsystems****

It can block, downgrade, or isolate.

****Rule 7 -- Mode-Switching Controller enforces isolation boundaries****

Prevents contamination or unsafe transitions.

5. Summary

This subsystem dependency graph:
shows how Claire's engines interact
proves the architecture is coherent and buildable
demonstrates clean separation of deterministic vs. connected logic
highlights safety and governance as first-class systems
satisfies acquisition-grade engineering diligence